

Lab 10 Spring WebFlux + WebClient

นายฐิติศักดิ์ ภูขยัน

Sec2

Pretty-print ☐

```
[{"id": "1", "name": "iPhone 15 Pro (673380035-5 SEC 2)", "category": "Electronics", "brand": "Apple", "stock": 50, "price": 39900.0, "discountType": "MEMBER", "discountedPrice": 35910.0}, {"id": "2", "name": "MacBook Air M3", "category": "Electronics", "brand": "Apple", "stock": 20, "price": 49900.0, "discountType": "NONE", "discountedPrice": 49900.0}, {"id": "3", "name": "Samsung Galaxy S24", "category": "Electronics", "brand": "Samsung", "stock": 30, "price": 29900.0, "discountType": "SEASONAL", "discountedPrice": 23920.0}]
```

Pretty-print ☐

```
{"id": "1", "name": "iPhone 15 Pro (673380035-5 SEC 2)", "category": "Electronics", "brand": "Apple", "stock": 50, "price": 39900.0, "discountType": "MEMBER", "discountedPrice": 35910.0}
```

Pretty-print ☐

35910.0

Pretty-print ☐

```
{"id": "1", "name": "iPhone 15 Pro (673380035-5 SEC 2)", "category": "Electronics", "brand": "Apple", "stock": 50, "price": 39900.0, "discountType": "MEMBER", "discountedPrice": 35910.0}
```

จัดรูปแบบ ☒

```
[
  {
    "id": "1",
    "name": "iPhone 15 Pro (673380035-5 SEC 2)",
    "category": "Electronics",
    "brand": "Apple",
    "stock": 50,
    "price": 39900,
    "discountType": "MEMBER",
    "discountedPrice": 35910
  },
  {
    "id": "2",
    "name": "MacBook Air M3",
    "category": "Electronics",
    "brand": "Apple",
    "stock": 20,
    "price": 49900,
    "discountType": "NONE",
    "discountedPrice": 49900
  },
  {
    "id": "3",
    "name": "Samsung Galaxy S24",
    "category": "Electronics",
    "brand": "Samsung",
    "stock": 30,
    "price": 29900,
    "discountType": "SEASONAL",
    "discountedPrice": 23920
  }
]
```

1. Reactive Programming กับ Blocking

Blocking Programming คือการทำงานที่ Thread ต้องหยุดรอผลลัพธ์ก่อนจึงทำงานต่อ เช่น การเรียก Database หรือ HTTP แล้วรอ response ทำให้ใช้ Thread มากและรองรับผู้ใช้จำนวนมากได้ไม่ดี

Reactive Programming คือการทำงานแบบ asynchronous เมื่อส่ง request แล้วไม่ต้องหยุดรอ Thread สามารถไปทำงานอื่นได้ เมื่อข้อมูลพร้อมจึงส่งผลลัพธ์กลับมา เหมาะกับงานที่มีหลาย request และการรับส่งข้อมูลแบบต่อเนื่อง

โปรเจกต์นี้ใช้ Spring WebFlux และ Reactor จึงคืนค่าเป็น **Mono** และ **Flux** และไม่มีการใช้ **.block()** หรือ code ที่ทำให้ Thread รอผลลัพธ์

2. Mono และ Flux

```
Mono<Product> getById(String id)
```

```
Mono<Product> save(Product product)
```

```
Mono<Double> getDiscountedPrice(String id)
```

ใช้แทนข้อมูล 0 ถึงหลายค่า เช่น:

```
Flux<Product> getAll()
```

```
Flux<Product> getByCategory(String category)
```

3. Operators ที่ใช้

map ใช้แปลงข้อมูลภายใน stream เช่น แปลง **Product** เป็นราคาหลังส่วนลด

```
getById(id)

    .map(Product::getDiscountedPrice);
```

ใช้กรองข้อมูล เช่น เลือกสินค้าที่มี category ตรงกัน

```
findAll()  
  
    .filter(product ->  
  
        product.getCategory().equalsIgnoreCase(category));
```

4. WebClient

WebClient ใช้เรียก REST API แบบ reactive และ non-blocking

client.get()

```
    .uri("/products/{id}", id)  
  
    .retrieve()  
  
    .bodyToMono(Product.class);
```

ความหมายของแต่ละ method:

.get() ใช้ส่ง HTTP GET

.post() ใช้ส่งข้อมูลใหม่

.delete() ใช้ลบข้อมูล

.uri() กำหนด URL และ path parameter

.bodyValue() ใส่ข้อมูลใน request body

.retrieve() รับ response จาก server

.bodyToMono() แปลง response เป็นข้อมูลหนึ่งรายการ

.bodyToFlux() แปลง response เป็นข้อมูลหลายรายการ

5. อธิบาย Code ของแต่ละส่วน

Repository

ProductRepository ใช้ ConcurrentHashMap เป็นฐานข้อมูลชั่วคราวในหน่วยความจำ รองรับ:

findByld() ค้นหาสินค้าตาม id

findAll() ดึงสินค้าทั้งหมด

save() บันทึกสินค้า

deleteByld() ลบสินค้า

findByCategory() กรองสินค้าตาม category

Service

ProductService เป็นส่วน business logic ทำหน้าที่:

ติดต่อกับ Repository

สร้าง UUID หากสินค้าไม่มี id

ใช้ map คำนวณราคาหลังส่วนลด

ใช้ flatMap ส่งข้อมูลไปบันทึก

จัดการกรณีค้นหาสินค้าไม่พบ

Controller

ProductController เป็น REST API ที่เปิด endpoint:

GET /products

GET /products/{id}

POST /products

DELETE /products/{id}

GET /products/category/{category}

GET /products/{id}/price

ทุก endpoint คืนค่าเป็น Mono หรือ Flux และไม่มี blocking code WebClient ProductWebClient ใช้เรียก endpoint ของ Controller ผ่าน HTTP แบบ reactive โดยคืนค่าเป็น Mono หรือ Flux เช่นเดียวกัน และสามารถ chain operator ต่อได้ทันที:

client.get()

.uri("/products/1/price")

.retrieve()

.bodyToMono(Double.class)

.map(price -> price + 100)

.defaultIfEmpty(0.0)

.subscribe(System.out::println);